

Witness-CL: Online Feature Reuse Under a Statistical Promotion Contract

Samuel Mausberg

AI-assisted research draft; author review and independent replication pending

Abstract

We study whether online feature reuse can justify its statistical admission cost without enumerating hidden worlds. Small numerical learners receive their own post-prediction scalar feedback from actual read-only SQLite queries; promoted feature syntax guides later search inside a supplied 84-monomial grammar. Frozen candidate policies face fresh paired betting tests with summable error spending. On sixteen frozen streams, reuse changes shared-novel reward by $+0.1224$ relative to no reuse (95% paired interval $[0.0316, 0.2132]$). Its preregistered transfer target is met, and dominance over all three simple controls at 4,096 SELECTs is not established. Lean 4.19.0 checks 77 accumulated statements, including deterministic finite-population accounting for erroneous promotions; the statistical validity argument is not formalized. A positive-mean counterexample proves that the fixed large bet can have poor admission probability even with unlimited samples. Given report routing and unchanged old policies supply structural retention. The study tests feature-search reuse and its opportunity cost, not native LLM benchmark superiority, open-ended abstraction, universal no-forgetting or alignment.

1 Problem and contribution boundary

The target is a deployed learner that improves across useful stateful episodes from its own experience, without an offline retraining phase and without losing valuable prior behavior. Preserving a transcript or weights is insufficient: the relevant object is future behavior under an explicitly declared task law. CL-Bench reports weak cross-instance reuse and a strong raw-history in-context learning (ICL) aggregate baseline [1]. AgentCL separates compositional transfer from persistence and exposes damage from irrelevant memory [2]. A useful solution must beat such controls after accounting for adaptation costs. Neither retaining history nor validating changes alone establishes that result.

Witness-CL’s supplied v4 ancestor learns hidden transducers inside a correctly declared finite class and checks immutable continuation policies. Our v5 and v6 extensions preserve its history and report rejected exploration hypotheses. In v6, two-probe planning improves re-

ward over a cheap zero-loss informative rule by 0.3938 per episode on 40 frozen streams, but loses 0.1948 to bounded optimism and uses $8.88\times$ the cheap rule’s planning time. Those results remain archived; they do not justify scaling enumerative planning as a solution.

This version asks a different bounded question: can useful feature reuse pay for fresh statistical auditing when there is no enumerated hidden-world class? Small numerical predictors learn after ordinary query episodes, promote frozen versions after paired testing, and reuse promoted feature syntax in later reports. Real SQLite queries replace simulated tool responses. The evaluator keeps report recipes private from the learner, but gives exact scalar feedback after ordinary predictions. This is supervised online feature search within a fixed grammar, not autonomous tool discovery or native LLM continual learning.

The contributions are an executable separation of fitting and fresh admission data; a query-budget comparison that charges the audit’s opportunity cost; a deterministic Lean accounting bridge for erroneous promotions; and explicit failure tests for the grammar, fixed routing and statistical power. The statistical machinery and component isolation have established antecedents. Any practical contribution must come from measured transfer over strong controls, not a novelty claim for attaching confidence tests to a memory library.

2 Existing approaches and unresolved gaps

Learned memory and executable libraries. ACE evolves contextual playbooks, ALMA searches memory designs, and EvoLib consolidates reusable abstractions [3, 5, 13]. These mechanisms are stronger than passive retrieval. Outer design search is nevertheless a cost and training distinction from solely online experience. TTHE evolves executable harnesses; its authors distinguish its same-batch adaptation/evaluation from future-batch generalization [4]. Recent computer-use work freezes versioned skill libraries within iterations and reports benefits as well as revision failures [14]. Neither frozen snapshots nor an evolving library is new here, and we have not reproduced these systems head to head.

Online parameters and isolation. Gradient constraints and orthogonal updates address interference in shared parameters [7, 10]; progressive networks isolate earlier components [8]. Such techniques require a matching task, memory and resource contract. Parameter isolation retains a function, but changing retrieval or dispatch can still change its use. Our report-specific frozen policies make that boundary explicit while assuming authentic report IDs. The numerical proposer may forget; deployed incumbents are separate objects. This does not show that a shared neural network retains arbitrary capabilities.

Model-based and statistical policy improvement. Predictive state representations model future observable experiments [12]. Deep SPI already joins representations, learned world models and safe online improvement [11]. Their abstraction and sampling assumptions remain substantive. High Confidence Policy Improvement (HCPI) separates candidate selection from testing expected-return constraints [15]. Its erratum corrects repeated testing to a bound growing with the number of tests [16]. A per-test confidence level is not a lifetime error budget.

Time-uniform confidence sequences and betting processes support valid optional stopping [6, 17]. Anytime off-policy evaluation offers a model-free route when only logged actions are available, with support, propensity and exogeneity obligations [9]. Conservative bandits protect cumulative arm means under their stochastic contract [18]; that is different from protecting each realized task. Thresholdout controls adaptive access to holdout data through a specified mechanism [19]. Simply renaming reused test examples is not such a mechanism. We choose fresh paired local execution because identical read-only starting instances are cheap, while explicitly measuring that cost. This convenience is an application assumption that a real deployment may not provide.

3 A bounded learner from its own episodes

3.1 Actual relational measurements

A context contains an in-memory SQLite database with 12–40 item rows and 3–8 group rows. Each item has four integer attributes in $[-3, 3]$ and a foreign key; each group has two weights in $[-2, 2]$. Two fixed ordered SELECTs expose both bounded tables. An immutable learner observation joins these rows by key, giving six values $z_i \in \mathbb{Z}^6$ per item. These measurements reveal the complete small tables; there is no active information acquisition in this phase.

The supplied typed grammar is

$$\mathcal{G} = \{m \in \mathbb{N}^6 : \|m\|_1 \leq 3\}, \quad |\mathcal{G}| = 84, \quad (1)$$

$$\phi_m(o) = \sum_i \prod_{k=1}^6 z_{ik}^{m_k}.$$

It contains count, six linear sums and all pairwise/cubic monomials. The evaluator’s private report is a one- or two-term integer combination of these aggregates. Independent SQL JOIN/SUM execution produces its target y . A prediction receives binary reward

$$R(\hat{y}, y) = \mathbf{1}\{|\hat{y} - y| \leq 10^{-6}(1 + |y|)\}. \quad (2)$$

Only after an ordinary prediction does the learner receive y . Report recipes, context seeds and evaluator phase labels are excluded from its API. Exact post-prediction feedback is a strong supervision assumption, not self-verification of an unobserved task answer.

3.2 Online fitting, search and reuse

For each public report token s , the learner retains its own bounded ordinary history H_s . A policy consists of immutable feature and coefficient tuples,

$$p_s(o) = \sum_{m \in A_s} \theta_{sm} \phi_m(o). \quad (3)$$

After each ordinary episode it fits small scaled least-squares systems; no offline phase or neural-model inference is used. The growing learner starts with seven features and adds at most five. Once observations outnumber active features by two, it scores up to 16 candidate features by correlation with the residual after projecting away the active span, then adds the best informative feature. A proposal can fail to find the relevant feature and cannot delete a bad early feature after reaching its cap.

Promoted nonlinear feature syntax enters a bounded shared bank. On a later report the bank changes candidate search order; coefficients are still fitted from that report’s own observations. Thus cross-report transfer is search reuse within a known basis, not transfer of an evaluator answer. The no-reuse arm removes only bank priority. A full-84-candidate growth arm tests whether a bank advantage merely substitutes for a restrictive search cap. All arms cache feature values already extracted from each ordinary observation.

Two strong controls use all legal ordinary history: three-step matching pursuit over the complete grammar and an 84-feature ridge regressor. The ridge penalty is 10^{-10} after column scaling. Small samples can leave this model underidentified even though the task is realizable; sparse recovery may exploit the much stronger sparsity prior sooner. We compare both audited and ungated sparse controls. An ungated control installs each ordinary proposal, making the audit’s delay and opportunity cost visible rather than holding those costs free.

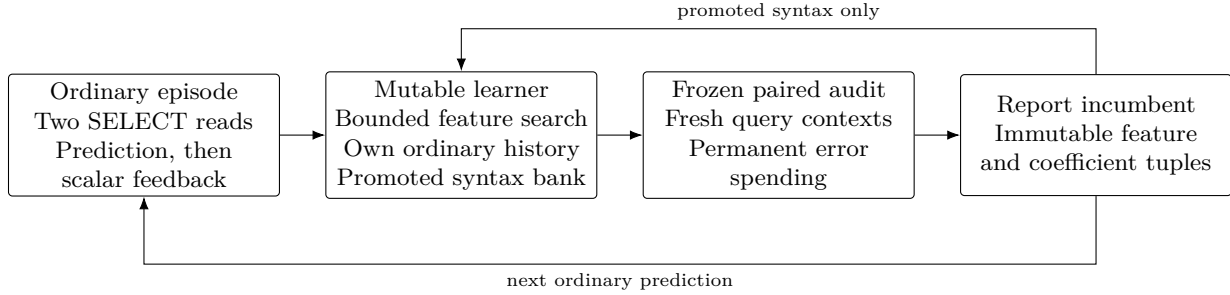


Figure 1: The v7 experiment. Fresh audit observations and targets never enter fitting. Public report dispatch and fixed read permissions belong to the trusted reference harness. Accepted policies change only their own report. Selection from the supplied grammar and the audit outcome can affect later proposals, which must face new audit samples.

3.3 Execution authority and accounting

Candidates never execute SQL. Trusted code sets up an ephemeral database, disables extensions, then enables query-only mode and an authorizer admitting fixed table reads and approved aggregate functions. VM steps, query time, rows and bytes are bounded. These checks do not isolate hostile Python running in the same process. Timing caps are cooperative execution controls, not hard real-time operating-system guarantees. Gold feedback and task dispatch are evaluator-owned.

Ordinary execution uses two measurement SELECTs and one gold SELECT. A paired audit repeats the measurements for each policy and shares one gold SELECT on the unchanged database, costing five. Database creation, inserts, PRAGMAs and commit are outside the named *post-setup SELECT* budget; their time is reported separately. Feature extraction, fitting, screening and policy execution are measured too. Numeric or serialized payload sizes are not actual resident memory or a claim of bounded lifelong storage.

4 What statistical admission establishes

4.1 Frozen candidates and summable error spending

Before audit j , freeze the candidate p_j , incumbent b_j , scope s_j and reward definition. Let paired fresh outcomes satisfy $D_t = R_t(p_j) - R_t(b_j) \in [-1, 1]$. The implemented process is

$$\begin{aligned}
 E_0 &= 1, & E_t &= \prod_{i=1}^t (1 + D_i/2), \\
 \alpha_j &= \frac{\alpha}{j(j+1)}, & \alpha &= .05.
 \end{aligned} \tag{4}$$

Acceptance requires $E_t \geq 1/\alpha_j$. A floating log-wealth calculation is checked by exact rational capital for the represented reward values, so upward rounding alone

cannot admit a candidate. Each started comparison spends alpha permanently, including abandoned or inconclusive comparisons. No audit exceeds 64 pairs and no run gate exceeds 1,024 starts. Complete hashes and globally unique sample IDs bind records, with actual data freshness an execution obligation.

Proposition 1 (Statistical argument; not Lean formalized). *Suppose each null comparison has $\mathbb{E}[D_t \mid \mathcal{F}_{t-1}] \leq 0$ at every pair, conditional on all information that selected its frozen policies. Then its probability of acceptance at any stopping time is at most α_j . Across an adaptive sequence of fresh comparisons, the probability of any false acceptance is at most α .*

Proof. The factors are nonnegative and $\mathbb{E}[E_t \mid \mathcal{F}_{t-1}] = E_{t-1}(1 + \mathbb{E}[D_t \mid \mathcal{F}_{t-1}]/2) \leq E_{t-1}$. Ville’s inequality bounds the crossing probability. Conditional validity and a union bound compose because $\sum_{j=1}^{\infty} 1/(j(j+1)) = 1$ [6, 17]. $\square$

Fresh independent draws from the same stationary scope distribution make the conditional mean a fixed policy-value difference. Under those assumptions the test supplies evidence of positive future scoped mean improvement. It does not protect every context or the failure tail. Under drift, a historical average conditional-mean interpretation is not a guarantee about tomorrow. Changed policies, routing or persistent state require a new valid comparison. The .05 allocation is per gate/run, not simultaneous across the benchmark’s many arms and random streams. An unchanged report is protected separately by immutable execution identity, not by passing another report’s statistical test.

The implementation’s journal preserves allocations through restoration and can reject rollback when supplied an external trusted monotone counter or journal head. An unkeyed hash chain cannot authenticate a rewritten journal or reward origin. The bounded local experiment does not implement a deployed durable consensus or crash-recovery service. Policy hashes freeze these small tuples, not arbitrary language-agent behavior outside the reference harness.

4.2 Validity can coexist with poor power

The first audit has $\alpha_1 = .025$. With constant difference d , 64 pairs can cross only if $(1 + d/2)^{64} \geq 40$, requiring approximately $d \geq .118664$. Perfect wins need ten pairs; a constant difference .2 needs 39. These are zero-variance diagnostics, not power guarantees for stochastic binary rewards.

A stronger limitation holds even without the cap. Let independent differences be +1 with probability .55 and -1 otherwise. Their mean is .1, but

$$\mathbb{E} \log(1 + D/2) = .55 \log 1.5 + .45 \log .5 < 0. \quad (5)$$

Moreover $q = .55\sqrt{1.5} + .45\sqrt{.5} < 1$, making $\sqrt{E_t}$ a supermartingale under this *positive-mean alternative*. Ville’s inequality then bounds eventual acceptance by $\sqrt{\alpha_j}$, below .159 for the first audit. The fixed large bet can therefore remain inconclusive on a better policy with substantial probability even after infinitely many pairs. A smaller prespecified bet has positive log growth for this example; predictable betting or a properly weighted fixed mixture is a standard next control [17]. This calculation exposes a power limitation, not a violation of null validity.

5 The checked deterministic retention bridge

Lean 4.19.0 checks 77 theorem declarations in the accumulated project, including 14 new finite-population accounting statements. No placeholders, custom source axioms or unexpected dependencies appear in the axiom inventory. Standard Lean logical axioms are recorded explicitly. The statistical supermartingale, probability model and sampling implementation above are *not* Lean proved.

For the new layer, a version stores C integer returns in $[0, B]$ on the same finite context population. Let $T(p)$ be their sum. Given a nonnegative integer per-promotion total allowance τ_j , its actual error indicator is

$$F_j = \mathbf{1}\{T(p_j) + \tau_j < T(p_{j-1})\}. \quad (6)$$

This event is computed from population values, not identified with an observed audit rejection. The learner need not know those values. The formal layer has no enumerated class of hidden transition tables or uninterpreted consistency oracle; finite return vectors express the evaluated policies’ actual values.

Theorem 2 (Lean-checked promotion accounting). *For every finite promotion history,*

$$T(p_0) \leq T(p_n) + \sum_{j=1}^n \tau_j + CB \sum_{j=1}^n F_j. \quad (7)$$

The same inequality applies to the suffix following any historical version.

For $C > 0$, division by C gives an expected-return bound under the uniform population distribution. Zero erroneous promotions and zero accumulated allowance imply that the final version’s population value is at least each historical version’s. No probability statement is smuggled into these premises. Nonzero allowances accumulate: totals $4 \rightarrow 3 \rightarrow 2$ each pass allowance one while losing two overall. Equal means also allow context loss: vectors $[1, 1]$ and $[0, 2]$ agree in total but regress on context zero, and under drift concentrated there. Both boundaries are checked examples.

An executable Lean fixture enumerates 2,916 two-promotion chains with two-context returns in 0..2 and allowances in 0..1. Independent Python comparisons check all 5,832 failure decisions and 8,748 historical bounds. This validates a bounded executable correspondence, not every Python implementation or a general probabilistic theorem. Authenticating that a real executor supplies the declared return vectors also remains outside Lean. Earlier interpreter/filter fixtures remain exact in their archive and coexist through additive imports and build targets.

6 Frozen evaluation design

6.1 Reports, streams and prediction timing

Four development seeds (80000–80003) precede a source/configuration freeze. Sixteen untouched evaluation seeds (81000–81015) are the independent statistical replicates. A stream samples four distinct $x_i w_j$ warm rules. Shared novel reports sum warm rules 0+1 and 2+3; two unshared reports use distinct cubic $x_i x_j w_k$ rules. Public report names are opaque hashes. Ordinary instances are shared across arms by cursor; audit and evaluation instances use disjoint seed namespaces. Seeds and phase labels never reach the learner.

The matched-example stream runs four warm reports round-robin for 24 examples each, four novel reports for 24 each, and four old reports for eight each: 224 predictions, each preceding its feedback. History cap 64 retains every ordinary example. Fresh old-report panels of 16 contexts each occur after warm and after novel learning with identical panel seeds and no learner updates. A final panel uses 32 fresh contexts for each report. These are all charged evaluation queries.

An audited candidate is considered every four ordinary examples, starting at four, if it gets at least .75 accuracy on its own history and at least one more history example correct than the incumbent. This screening may overfit and is never admission evidence. A successful audit changes only its report’s policy. Audit outcomes do not train weights or enter ordinary history. All later candidates, including those influenced by previous acceptance, face fresh pairs.

The primary transfer estimand is mean prequential reward over the first 24 examples of the two shared novel reports, paired by seed between reuse and no reuse. The first eight examples are a predeclared negative control: feature-bank search cannot yet affect an installed policy. Unshared transfer, retention, final return and control comparisons are reported separately. The primary hypothesis needs at least .05 mean gain with a positive paired 95% lower bound.

6.2 Audit opportunity cost

Matching ordinary examples gives different actual query costs. A second protocol gives every arm 4,096 post-setup SELECTs and reserves 768 for the final panel. At most 3,328 remain for ordinary episodes and audits. Ungated learners spend saved audit queries on more ordinary feedback from the same stream prefix; after episode 224 the stream cycles through all eight reports. No arm skips warm episodes to reach later reports. An audit stops inconclusive if fewer than five queries remain; an ordinary episode starts only with at least three. Unreached reports remain at the zero anchor and still appear in the final panel. All histories use cap 256, sufficient for every possible ordinary exposure. There are no intermediate panels in this companion.

The fixed practical control set is audited sparse history, ungated sparse history and ungated full ridge. Practical usefulness requires at least .05 final-panel gain and a positive paired 95% lower bound against *each* control, with no old-policy mutation. This conjunction prevents selecting an easy comparator after seeing outcomes. All paired comparisons are retained. We use stream-level Student intervals; contexts within a stream are not independent experimental replicates. The criterion combines an estimated gain of .05 with evidence of positive gain; it does not establish a 95% lower guarantee of a gain at least .05. All experimental intervals are nominal Student intervals; sixteen bounded stream scores do not establish exact coverage. Other estimates and observed rankings are descriptive, without a blanket familywise confidence claim.

7 Results and independent replay

In the companion rows of Table 1, Shared averages only the ordinary examples actually reached among each shared report’s first 24; the budget usually stops before all 24 arrive. It is a censored-prefix descriptive score, not the matched protocol’s complete-window transfer estimand. The feedback-free final macro score is the companion’s prespecified comparison.

The primary difference between audited reuse and no reuse is +0.12240 reward (95% paired interval [0.03158, 0.21321]). The predeclared five-point transfer

target is met. This comparison isolates a bounded search-order mechanism, not general representation learning.

At the common SELECT ceiling, reuse minus sparse history, audited is -0.52563 (95% interval [-0.55610, -0.49517]). At the common SELECT ceiling, reuse minus sparse history, ungated is -0.52563 (95% interval [-0.55610, -0.49517]). At the common SELECT ceiling, reuse minus full ridge, ungated is -0.52563 (95% interval [-0.55610, -0.49517]). The conjunction required for practical dominance is not satisfied. All fixed comparisons are reported; the best observed control is not selected as a new hypothesis.

Matched old-report panels show 0 changed policy/prediction entries during novel learning. Those execution paths are unchanged by construction under fixed public dispatch. This supports the implementation invariant, not inferred task identity or arbitrary-distribution retention.

An evaluator-only diagnostic finds all true component monomials in the final selected features for 34.4% of shared novel reports, while 71.9% reach the twelve-feature cap. These hidden recipes are examined only after execution. Irreversible early feature additions can exhaust the cap with spurious choices; larger candidate search alone need not repair that failure. This is a diagnostic inference from the saved feature sets, not an additional confirmatory hypothesis.

The holdout records 78,081 ordinary episodes, 61,790 audit pairs and 61,440 panel contexts, totaling 727,513 post-setup SELECTs. Raw policy versions, observations, context seeds, gate journals and all failures are retained. The source/configuration freeze precedes these streams; four development streams remain separate.

Executable validation. The standard Python suite passes 656 tests with 8 optional native-environment skips. C++ checks pass 432 reference cases, 200 randomized continuation cases, and 400 Python/C++ pairs with undefined-behavior sanitization. The Lean audit checks 77 accumulated statements with no unexpected axioms. Independent standard-library replay passes every held-out arm/run, reconstructing visible rows, computing gold targets with separate SQL, evaluating serialized policies independently, and checking exact gate capital, permanent alpha, policy transitions, complete panels and budget prefixes. Its receipt binds each raw file and the source/configuration freeze. It verifies recorded consistency and accounting; it does not independently measure original timings or authenticate historical execution.

8 What remains unsolved

The experiment gives stable scope identifiers, full small-table observations, exact scalar feedback, cheap independent instances and a realizable sparse polynomial grammar. These assumptions remove latent routing,

Table 1: Frozen 16-stream evaluation. All values are means across streams. Shared is prequential reward on the first 24 examples of two shared novel reports. Final is the eight-report macro mean on fresh feedback-free panels. Matched arms each receive 224 ordinary examples; companion arms spend at most 4,096 post-setup SELECTs, including final evaluation. Run time covers fitting, screening, queries and setup; construction and final artifact export are outside that timer. Timing is descriptive on this machine.

Protocol	Method	Shared	Final	Ordinary n	Audit pairs	SELECTs	Run s
Matched	Grow + reuse, audited	0.174	0.681	224.0	613.2	4889.9	1.36
	Grow, no reuse, audited	0.052	0.734	224.0	623.1	4939.3	1.38
	Sparse history, audited	0.513	1.000	224.0	123.4	2440.9	0.87
	Sparse history, ungated	0.547	1.000	224.0	0.0	1824.0	0.74
	Full ridge, ungated	0.003	0.000	224.0	0.0	1824.0	1.84
	Grow all 84, audited	0.073	0.788	224.0	631.4	4981.2	1.42
Budget	Grow + reuse, audited	0.014	0.474	138.1	582.6	4095.3	0.98
	Grow, no reuse, audited	0.014	0.474	139.8	581.6	4095.1	0.99
	Sparse history, audited	0.513	1.000	903.6	123.2	4094.8	3.20
	Sparse history, ungated	0.547	1.000	1109.0	0.0	4095.0	3.48
	Full ridge, ungated	0.003	1.000	1109.0	0.0	4095.0	7.59
	Grow all 84, audited	0.015	0.512	136.6	583.4	4095.1	1.02

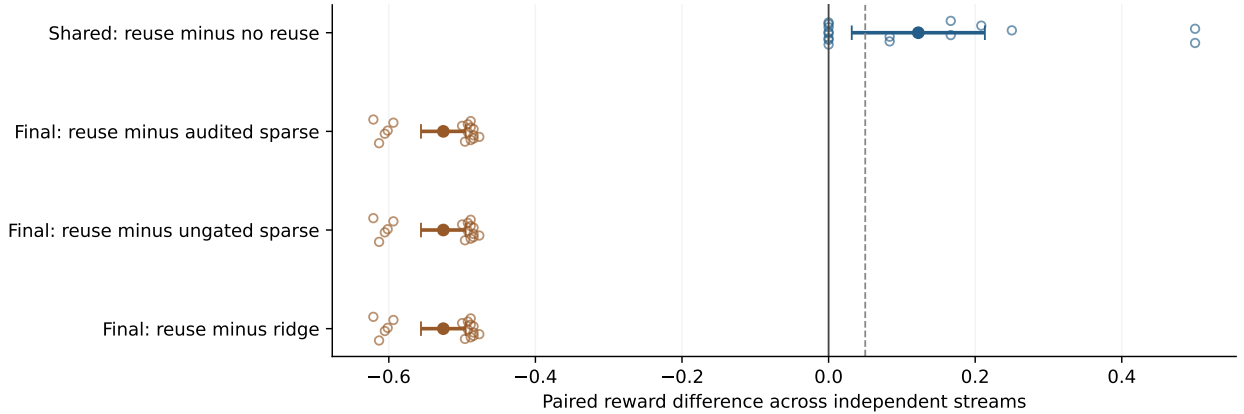


Figure 2: Seed-level paired differences (open points), their means and nominal 95% Student intervals. The top row uses matched shared-novel prequential reward; the remaining rows use the equal-SELECT companion final-panel reward. Zero and the predeclared .05 target are shown. Contexts within a stream are not statistical replicates.

scarce feedback, non-resettable actions and most representation uncertainty. A win here would be a mechanism result, not the requested stateful benchmark solution. A loss to simple sparse recovery is evidence against this instance of useful reuse. The 84-candidate ablation distinguishes search-order savings from an intrinsically better representation. A stronger grammar prior can make an expensive memory system unnecessary even if its contents are correct.

Auditing changes the problem too. It pays for evidence that ordinary deployment may not supply, excludes its labels from fitting, and can stall useful changes under finite budgets or the fixed bet’s low power. Retention follows authentic routing and immutability for untouched reports; those must be learned or independently justified in a native agent. A finite registry with growing history is not a fixed-memory lifelong learner. Noise and degree-four parity tests expose rather than solve misspecification.

No check provides alignment or future protection under arbitrary environment drift.

A falsifiable next mechanism. A more useful hypothesis is that a learned typed library can reduce later tool work on tasks whose grammar is not enumerated as complete aggregate features. Use one fixed, permitted backbone to propose bounded read-only query fragments from its own ordinary traces; freeze the fragment interpreter and permissions. Learn applicability from public schema and observed outputs, withholding hidden report identity. Store executable fragments with explicit preconditions and versioned evidence rather than memorized task answers. A separate resettable track can compare admission with a prespecified small-bet mixture, auditing each execution path an update can change. That track requires authoritative feedback and fresh paired tasks. Such a design joins known components; novelty

and usefulness would require new evidence, not the architecture’s name.

First measure throughput on at most four development streams with bounded local inference, record all proposal/audit calls and size a feasible budget. Then freeze model, prompts, selectors, token caps, tool permissions and seeds. Compare raw legal-history ICL, a strong evolving-library method, exact memoization, ungated proposals and the learned library on untouched CL-Bench-compatible resettable tasks and AgentCL compositional streams. Preserve each native benchmark’s actual information contract. AgentCL excludes ground-truth guidance during memory construction: even using gold scores only to accept a memory changes that contract. Such admission belongs in an explicitly augmented-feedback track, not a claimed native AgentCL result [2]. Give each method the same actual inference and environment budget; unused audit budget must be available to the controls. Track future ordinary reward and feedback-free old-task panels. No such native run is claimed in this version.

Require .05 normalized future gain over every fixed strong control, positive paired lower intervals over independent streams, and an old-task margin no worse than .02 under a declared total error budget. Kill the proposal if gains vanish under authentic task-identity removal, equal resource accounting or full-history controls; if library growth mainly memorizes answers; or if most useful proposals cannot be audited before the budget expires. Any reproducible data leakage, missing protected path or reset error-spending bug defeats the corresponding statistical claim. If fresh pairs are unavail-

able, move to an explicit supported logging design and off-policy inference rather than treating deterministic logs as counterfactual evidence [9].

Resources and safety boundary. The completed numerical phase runs on one CPU thread per BLAS pool with local ephemeral databases; it has no new LLM calls, deployments or paid compute. Native throughput must be measured before assigning GPU-hours. The existing integration contract is pinned and tested, but that is not native benchmark performance. Future model work must use a permitted standard model and controlled local tools. The user’s removed background automation remains removed. This phase does not schedule or authorize unattended deployment.

Reproducibility and disclosure

The private repository preserves the supplied Git history, release tags, prior raw data and the v6 paper under `paper/v6`. Current source, tests, complete gate journals, policy versions, context seeds, experiment freeze, independent replay, theorem/axiom inventory and generated tables accompany this draft. Reproduction commands and exact resource semantics appear in the README and `docs/v7/EVALUATION.md`. The probability proof above is a conventional mathematical argument; only the specified deterministic statements are checked in Lean. This is an AI-assisted research draft for Samuel Mausberg, with author review and independent replication pending. The general native continual-learning problem remains open.

References

- [1] Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, Joseph E. Gonzalez. Continual Learning Bench: Evaluating Frontier AI Systems in Real-World Stateful Environments. 2026. <https://arxiv.org/abs/2606.05661>.
- [2] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, Yu Su. AgentCL: Toward Rigorous Evaluation of Continual Learning in Language Agents. 2026. <https://arxiv.org/abs/2606.02461>.
- [3] Qizheng Zhang, et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. 2025. <https://arxiv.org/abs/2510.04618>.
- [4] Jun Nie, et al. TTHE: Test-Time Harness Evolution. 2026. <https://arxiv.org/abs/2607.08124>.
- [5] Yiming Xiong, Shengran Hu, Jeff Clune. Learning to Continually Learn via Meta-learning Agentic Memory Designs. 2026. <https://arxiv.org/abs/2602.07755>.
- [6] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, Jasjeet Sekhon. Time-uniform, Nonparametric, Nonasymptotic Confidence Sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021. <https://arxiv.org/abs/1810.08240>.
- [7] David Lopez-Paz, Marc’Aurelio Ranzato. Gradient Episodic Memory for Continual Learning. 2017. <https://arxiv.org/abs/1706.08840>.
- [8] Andrei A. Rusu, et al. Progressive Neural Networks. 2016. <https://arxiv.org/abs/1606.04671>.
- [9] Ian Waudby-Smith, Lili Wu, Aaditya Ramdas, Nikos Karampatziakis, Paul Mineiro. Anytime-valid Off-policy Inference for Contextual Bandits. 2022. <https://arxiv.org/abs/2210.10768>.
- [10] Mehrdad Farajtabar, Navid Azizan, Alex Mott, Ang Li. Orthogonal Gradient Descent for Continual Learning. In *International Conference on Artificial Intelligence and Statistics*, 2020. <https://proceedings.mlr.press/v108/farajtabar20a.html>.
- [11] Florent Delgrange, Raphael Avalos, Willem Röpke. Deep SPI: Safe Policy Improvement via World Models. In *International Conference on Learning Representations*, 2026. <https://arxiv.org/abs/2510.12312>.
- [12] Satinder Singh, Michael James, Matthew Rudary. Predictive State Representations: A New Theory for Modeling Dynamical Systems. In *Proceedings of the Twentieth Conference on Uncertainty in Artificial Intelligence*, 2004. <https://arxiv.org/abs/1207.4167>.
- [13] Weijia Xu, Alessandro Sordoni, Chandan Singh, Zelalem Gero, Michel Galley, Xingdi Yuan, Jianfeng Gao. Test-Time Learning with an Evolving Library. 2026. Version 2, July 14. <https://arxiv.org/abs/2605.14477>.
- [14] Longtao Hu, Xiao Liang, Linchao Zhu. From Interaction Traces to Persistent Skills: Online Evolution for Computer-Use Agents. 2026. Submitted September 4. <https://arxiv.org/abs/2609.04869>.
- [15] Philip S. Thomas, Georgios Theodorou, Mohammad Ghavamzadeh. High Confidence Policy Improvement. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015. <https://proceedings.mlr.press/v37/thomas15.pdf>.
- [16] Philip S. Thomas, Georgios Theodorou, Mohammad Ghavamzadeh. Errata for High Confidence Policy Improvement. 2017. <https://people.cs.umass.edu/~pthomas/papers/Thomas2015bErrata.pdf>.
- [17] Ian Waudby-Smith, Aaditya Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(1):1–27, 2024. <https://doi.org/10.1093/jrsssb/qkad009>.
- [18] Yifan Wu, Roshan Shariff, Tor Lattimore, Csaba Szepesvári. Conservative Bandits. In *Proceedings of the 33rd International Conference on Machine Learning*, 2016. <https://proceedings.mlr.press/v48/wu16.pdf>.
- [19] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, Aaron Roth. Generalization in Adaptive Data Analysis and Holdout Reuse. 2015. <https://arxiv.org/abs/1506.02629>.